

# Lab-7.3

D.HIMESH  
2303A52368  
Batch-38

## Task 1: Fixing Syntax Errors

### Scenario

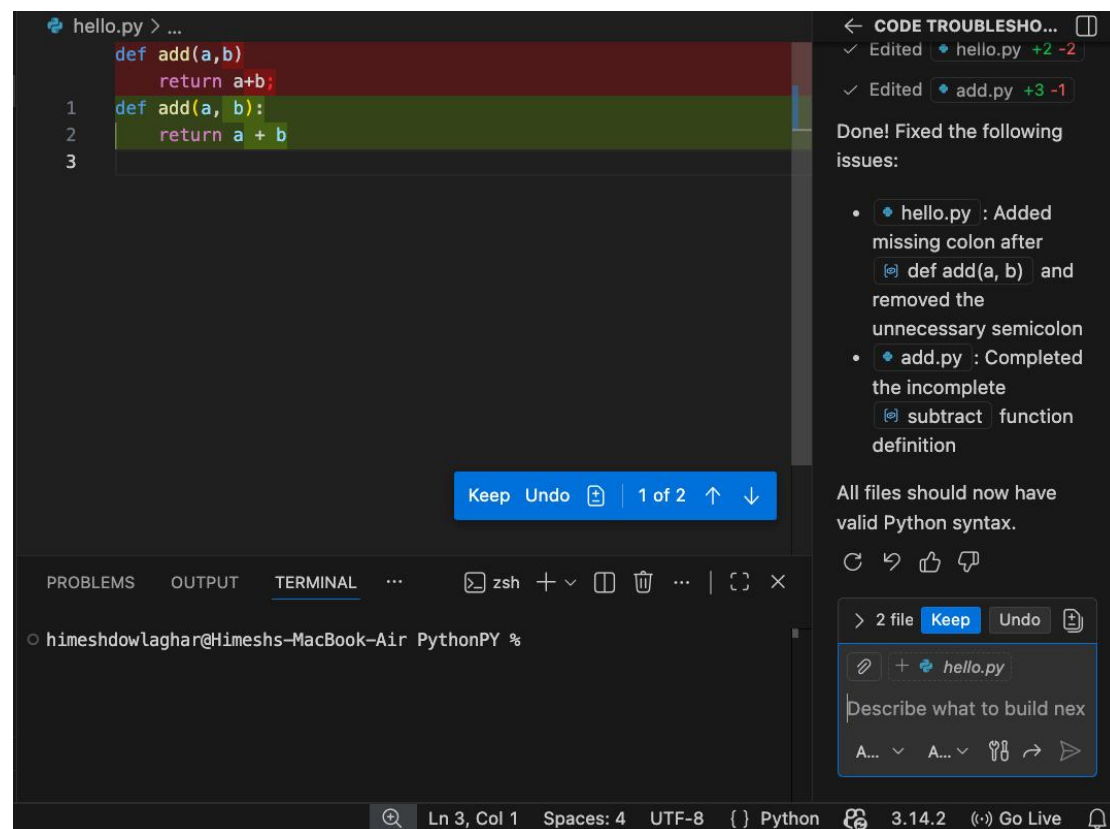
You are reviewing a Python program where a basic function definition contains a syntax error.

Given Code:

```
def add(a,b)  
    return a+b
```

Corrected:

```
def add(a,b):  
    return a+b
```



## Task 2: Debugging Logic Errors in Loops

### Scenario

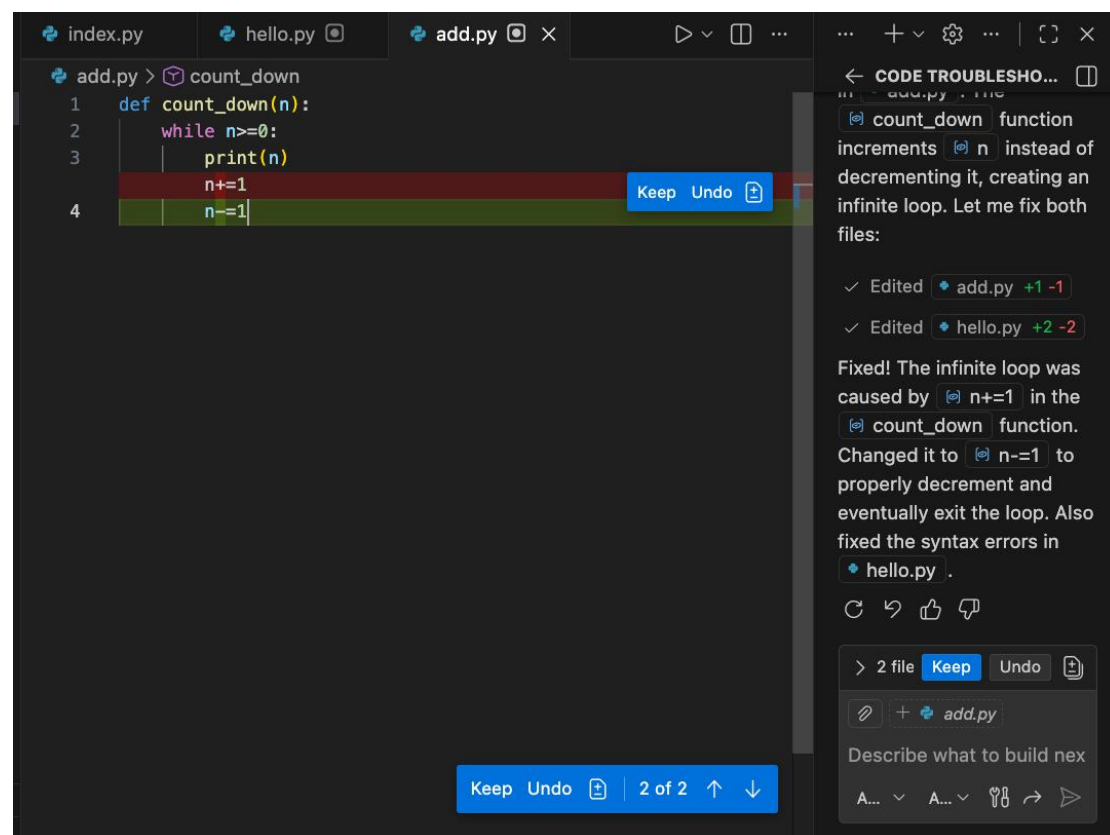
You are debugging a loop that runs infinitely due to a logical mistake.

Given Code:

```
def count_down(n):  
    while n>=0:  
        print(n)  
        n+=1
```

Corrected Code:

```
def count_down(n):  
    while n>=0:  
        print(n)  
        n-=1; //corrected here the infinite loop
```



### Task 3: Handling Runtime Errors (Division by Zero)

#### Scenario

A Python function crashes during execution due to a division by zero error.

#### Given Code:

```
def divide(a,b):  
    return a/b  
print(divide(12,0))
```

#### Corrected Code:

```
def divide(a,b):  
    if b == 0:  
        return "Error: Division by zero"  
    return a / b  
print(divide(10,0))
```

The screenshot shows a code editor with a dark theme. The code is as follows:

```
def divide(a,b):  
    return a/b  
print(divide(12,0))  
  
1 def divide(a, b):  
2     if b == 0:  
3         return "Error: Cannot divide by zero"  
4     return a / b  
5  
6 print(divide(12, 2))
```

Below the code, there are buttons for "Keep" and "Undo".

On the right side, there is a chat window with the following text:

Can I help you with the error in your code? The error is a runtime error, which causes a runtime error. Let me fix it:

✓ Edited add.py +6 -3

Fixed! The semantic error was a division by zero ( `divide(12, 0)` ). Added a zero-check in the function and changed the call to `divide(12, 2)` to avoid the error.

At the bottom of the chat window, there are buttons for "Keep" and "Undo", and a text input field with the placeholder "Describe what to build next".

## Task 4: Debugging Class Definition Errors

### Scenario

You are given a faulty Python class where the constructor is incorrectly defined.

Given Code:

```
class Rectangle:
def __init__(length,width):
self.length=length
self.width=width
```

Corrected Code

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
```

The screenshot shows a code editor with a Python class definition. The original code has an error in the `__init__` method signature, which is missing the `self` parameter. The corrected code adds `self` as the first argument. A sidebar on the right provides a detailed error message and a 'fix the code' button. The error message states: 'Found the error: the `__init__` method is missing the `self` parameter. Let me fix it:'. Below this, it shows the edited file 'add.py' with changes '+3 -3'. The fix is described as: 'Fixed! The issue was that `__init__` was missing the `self` parameter as the first argument. Added it and also improved spacing for consistency.' At the bottom, there are buttons for 'Keep', 'Undo', and '1 of 1'.

```
1 class Rectangle:
2     def __init__(length,width):
3         self.length=length
4         self.width=width
5
```

fix the code

✓ Read Python file

Found the error: the `__init__` method is missing the `self` parameter. Let me fix it:

✓ Edited `add.py` +3 -3

Fixed! The issue was that `__init__` was missing the `self` parameter as the first argument. Added it and also improved spacing for consistency.

< 1 file Keep Undo

add.py:1-5

Describe what to build next

A... A... < >

Ln 5, Col 1 (112 selected) Spaces: 4 UTF-8 { } Python 3.14.2 Go Live

## Task 5: Resolving Index Errors in Lists

### Scenario

A program crashes when accessing an invalid index in a list.

Given Code:

```
numbers=[1,2,3]
print(numbers[5])
```

Corrected Code:

```
numbers = [1, 2, 3]

# Method 1: Exception handling
try:
    print(numbers[5])
except IndexError:
    print("Error: Index out of bounds")
```

```
# Method 2: Bounds checking
index = 5
if 0 <= index < len(numbers):
    print(numbers[index])
else:
    print(f"Error: Index {index} is out of bounds for list of length {len(numbers)}")
```

```
# Method 3: Safe get with default value
print(numbers[2] if 2 < len(numbers) else "Index not found")
```

```
numbers = [1, 2, 3]

# Method 1: Exception handling
try:
    print(numbers[5])
except IndexError:
    print("Error: Index out of bounds")

# Method 2: Bounds checking
index = 5
if 0 <= index < len(numbers):
    print(numbers[index])
else:
    print(f"Error: Index {index} is out of bounds for list of len {len(numbers)}")

# Method 3: Safe get with default value
print(numbers[2] if 2 < len(numbers) else "Index not found")
```

✓ Edited • add.py +16 -1

Added three safe access methods:

1. **Exception Handling** - Catches `IndexError` if index is out of bounds
2. **Bounds Checking** - Validates index before accessing with `0 <= index < len(numbers)`
3. **Conditional Access** - Uses ternary operator with default fallback

The code now handles invalid indices gracefully without crashing.

↻ ↶ 👍 🗨

🔗 + add.py

5

A... ▾ A... ▾ 🏠 ↶ ➤



Ln 14, Col 35

Spaces: 4

UTF-8

{ } Python



3.14.2

(Go Live)

